

课 程 设 计 报 告

姓　名	张瑞淇
学　号	20240264

课程名称 软件综合实践

设计题目 电影票房数据可视化系统

课程教师 王文鑫

设计起止日期 2021.7.5-2021.7.23

评　　分 _____

大数据与软件学院

目 录

1. 绪论 1
1.1. 项目背景 1
1.2. 项目目标 1
2. 需求分析 1
2.1. 系统功能模块 1
2.1.1. 数据采集模块 1
2.1.2. 数据展示模块 2
2.1.3. 可视化分析模块 2
2.1.4. 智能交互模块 2
2.1.5. 系统功能模块图 3
2.2. 主要功能描述 3
2.3. 用例图 5
3. 数据库设计 5
3.1. 需求分析 5
3.2. 数据库概念结构设计 6
3.3. 数据库逻辑结构设计 6
3.4. 数据库物理设计 7
4. 系统设计 8
4.1. 系统架构 8
4.2. 后端 API 设计 9
4.3. 智能助手时序图 10
4.4. 类图 11
4.5. 缓存设计说明 11
4.6. 预测模块设计 12
5. 编码实现 13
5.1. 开发环境 13
5.2. 运行环境 13
5.3. 关键代码分析 13

5.4. 主要界面截图 17
6. 程序调试情况 21
6.1. 数据采集阶段 21
6.2. 后端开发阶段 22
6.3. 前端开发阶段 22
6.4. 智能助手阶段 23
7. 系统总结 23
7.1. 系统特点 23
7.2. 存在问题 24
7.3. 改进意见 24
8. 课程设计总结 24
9. 参考文献 25

1. 绪论

1.1. 项目背景

随着电影产业的快速发展，全球电影市场数据量呈指数级增长。TMDB（The Movie Database）作为全球最大的开放电影数据库平台，提供了海量的电影元数据；猫眼电影作为国内领先的电影票务平台，积累了丰富的中国本土票房数据。然而，这些数据分散在不同平台，缺乏统一的整合与可视化分析工具。

本项目旨在构建一个电影票房数据可视化系统，通过整合 TMDB 与猫眼双源数据，利用 ECharts 实现多维度可视化分析，并引入大语言模型接口，为用户提供自然语言驱动的智能推荐与票房预测服务，帮助影视从业者、投资者和影迷更直观地理解电影市场规律。

1.2. 项目目标

- 采集并清洗 TMDB 和猫眼的电影数据，构建完整的电影信息数据库
- 提供电影列表查询、多维度筛选、排序及无限滚动浏览功能
- 实现上映趋势、票房趋势、类型统计、评分分布四个维度的可视化分析
- 集成大语言模型，实现自然语言驱动的电影推荐与票房预测

2. 需求分析

2.1. 系统功能模块

本系统划分为四个核心功能模块，各模块协同工作，共同构成完整的电影票房数据可视化平台。

2.1.1. 数据采集模块

该模块负责从外部数据源获取原始电影数据，是系统的数据基础。包含两个子模块：TMDB 爬虫通过调用 TMDB 官方 API 获取电影基本信息、评分、海报 URL 和演员信息，需要处理 API 限流问题；猫眼爬虫负责抓取中国本土票房数据，需要应对字体反爬、Cookie 验证、IP 频率限制等多层反爬策略。采集后的数据经过清洗和去重后存入 MySQL 数据库，共采集 10,294 条有效记录，涵盖 2002 年至 2026 年的电影信息。

2.1.2. 数据展示模块

该模块负责将数据库中的电影数据以直观的方式呈现给用户。列表子模块以卡片网格形式展示电影，每张卡片包含海报、片名、评分和年份等关键信息，无海报时自动显示"无封面"占位文字。筛选子模块支持按类型和按年份进行多选筛选，满足不同维度的数据过滤需求。排序子模块支持按评分和按票房两种排序方式，方便用户从不同角度浏览数据。分页子模块采用滚动触底无限加载方案，每次加载 24 条记录，配合图片懒加载，在万余条数据规模下仍能保持流畅的浏览体验。

2.1.3. 可视化分析模块

该模块基于 ECharts 实现四个维度的数据可视化分析。上映趋势页面通过柱状图与折线图组合展示各年份电影数量变化，右侧侧边栏列出每年具体统计数字，便于观察电影产出量的年度波动。票房趋势页面展示各年份平均每部电影的票房（而非总票房），有效消除年份间电影数量差异带来的数据偏差，柱状图采用绿色渐变配色，右侧同步显示每年电影数量和平均票房金额。类型统计页面通过环形图和柱状图展示各类型电影分布，采用黄色系渐变配色，显示全部类型而非仅 TOP N，环形图加载时带有旋转展开动画效果。评分分布页面采用横向条形图展示按 0.5 分档聚合的评分分布，Y 轴为评分档位，X 轴为电影数量，标签显示在条形右侧便于阅读，配色采用蓝色渐变且数量越多颜色越深。

2.1.4. 智能交互模块

该模块是大语言模型与系统功能的结合点，也是本项目的核心亮点之一。推荐子模块通过调用硅基流动 DeepSeek API 解析用户的自然语言输入，提取类型、年份范围、最低评分、关键词等筛选条件，动态构建 SQL 查询并返回符合条件的电影列表。系统还支持"类似 XX"的模糊推荐——先查询参考电影的类型和评分特征，再基于这些特征检索相似电影。预测子模块基于历史数据建立简易预测模型，综合考虑类型系数、预算规模和年份增长趋势等因素估算票房，预测结果以卡片形式展示预算、预测票房、投资回报率和风险评估等关键指标。当 API 调用失败时，系统自动降级到关键词匹配的备用解析方案，保证基础服务始终可用。

2.1.5. 系统功能模块图



图 1 系统功能模块图

2.2. 主要功能描述

功能模块	子功能	描述
数据采集	TMDB 爬虫	调用 TMDB API 补全海报 URL 和演员信

功能模块	子功能	描述
		息，处理限流问题
数据采集	猫眼爬虫	应对字体反爬、Cookie 验证、IP 频率限制等反爬策略
数据展示	电影总览	卡片网格展示，支持类型/年份筛选，评分/票房排序，无限滚动
数据展示	电影详情	模糊背景弹窗，展示完整信息、演员列表、词云
可视化分析	上映趋势	柱状图+折线图展示各年份电影数量变化
可视化分析	票房趋势	平均票房柱状图+折线图，按年份排序
可视化分析	类型统计	环形图+柱状图（黄色系），显示全部类型
可视化分析	评分分布	横向条形图，按 0.5 分档聚合
智能交互	电影推荐	自然语言解析用户需求，返回符合条件的电影列表
智能交互	票房预测	基于类型/预算/年份估算票房，展示 ROI 和风险评估
系统管理	夜间模式	CSS 变量切换，localStorage 持久化
系统管理	缓存管理	内存缓存 TTL 过期，连接池管理

2.3. 用例图



图 2 用例图

本系统的用例图展示了两种主要用户角色与系统功能之间的关系。

普通用户 是系统的主要使用者，可以浏览电影列表、按类型或年份筛选电影、按评分或票房排序、滚动触底无限加载更多电影、点击卡片查看包含完整信息和词云的电影详情弹窗，并在智能助手页面通过自然语言对话获得电影推荐和票房预测服务。

数据分析师 在普通用户功能基础上，更侧重于数据洞察和趋势分析，通过上映趋势、票房趋势、类型统计、评分分布四个可视化页面进行多维度数据观察，并利用侧边栏的详细统计列表进行数据对比。

两种角色共用系统的核心数据资源，但数据分析师对可视化分析模块有更强的使用需求，普通用户则更侧重浏览和智能交互体验。系统当前版本暂未开放数据导出功能，但已在后续优化计划中。

3. 数据库设计

3.1. 需求分析

系统需要存储电影的核心信息，包括：片名、上映日期、预算、票房、片长、评分、投票数、类型、简介、年份、海报 URL、演员信息等。数据来源于 TMDB 和猫眼双源，需要按"片名+年份"维度去重，类型字段需从中英文混杂统一转换为标准化中文。

3.2. 数据库概念结构设计

本系统核心实体为 tmdb_movies（电影信息表），存储从 TMDB 和猫眼双源采集的电影完整数据。该实体包含电影基本信息（片名、上映日期、预算、票房、片长）、评分信息（评分、投票数）、类型信息（原始类型、标准化类型）以及展示信息（简介、海报 URL、演员 JSON 数据）。各字段均允许为空，以兼容不同数据源的数据缺失情况。数据采集时按 片名 + 年份 组合进行去重，确保同一电影不重复入库，且不含 id 主键字段。实体间无关联表，属于单实体系统。

3.3. 数据库逻辑结构设计

将概念实体转换为 MySQL 数据表，定义各字段的名称、数据类型及索引策略：

字段名	类型	允许 NULL	说明
片名	longtext	NO	电影名称，与年份组合作为去重依据
上映日期	longtext	YES	电影公映日期
预算	bigint	YES	制作预算（美元），缺失或为 0 表示数据不可用
票房	bigint	YES	全球票房收入（美元），为 0 时前端显示"暂无票房"
片长	double	YES	播放时长（分钟），支持半分钟精度
评分	double	YES	TMDB 综合评分（0-10），用于排序和筛选
投票数	bigint	YES	TMDB 评分参与人数，反映评分可信度
类型	longtext	YES	原始类型字段，中英文混杂，用于数据溯源
简介	longtext	YES	剧情简介，自动翻译为中文并缓存 24 小时

字段名	类型	允许 NULL	说明
年份	double	YES	上映年份，已建立索引，用于趋势分析和年份筛选
poster_url	longtext	YES	海报 URL，http 自动替换为 https，缺失时显示"无封面"
类型_标准化	longtext	YES	标准化中文类型（如"剧情,动作"），逗号分隔，用于类型筛选和统计
演员	text	YES	演员信息 JSON 数组（name/character/profile_path），后端仅提取 name 传给前端

上述字段中，片名 + 年份 组合作为记录的去重依据，数据采集时按此组合判断记录是否已存在，避免重复入库。

索引字段	索引类型	说明
年份	BTREE	优化年份趋势查询和筛选性能
评分	BTREE	优化按评分排序查询性能
票房	BTREE	优化按票房排序查询性能

3.4. 数据库物理设计

采用 utf8mb4 字符集支持完整 Unicode，年份、评分、票房分别建立 BTREE 索引以优化趋势查询和排序性能，存储引擎使用 InnoDB 支持事务和行级锁。

```
CREATE DATABASE IF NOT EXISTS movie_db DEFAULT CHARSET utf8mb4;
USE movie_db;
CREATE TABLE tmdb_movies (
    片名 LONGTEXT NOT NULL,
    上映日期 LONGTEXT,
    预算 BIGINT,
```

```
    票房 BIGINT,
    片长 DOUBLE,
    评分 DOUBLE,
    投票数 BIGINT,
    类型 LONGTEXT,
    简介 LONGTEXT,
    年份 DOUBLE,
    poster_url LONGTEXT,
    类型_标准化 LONGTEXT,
    演员 TEXT,
    INDEX idx_year (年份),
    INDEX idx_rating (评分),
    INDEX idx_revenue (票房)
);
```

4. 系统设计

4.1. 系统架构

本系统采用前后端分离架构。前端通过 Fetch API 调用后端 RESTful 接口获取 JSON 数据并渲染页面，后端负责业务逻辑处理、数据库查询和缓存管理，前端负责界面交互、状态管理和图表渲染。前后端通过标准化 JSON 数据格式进行通信，职责清晰、便于维护。

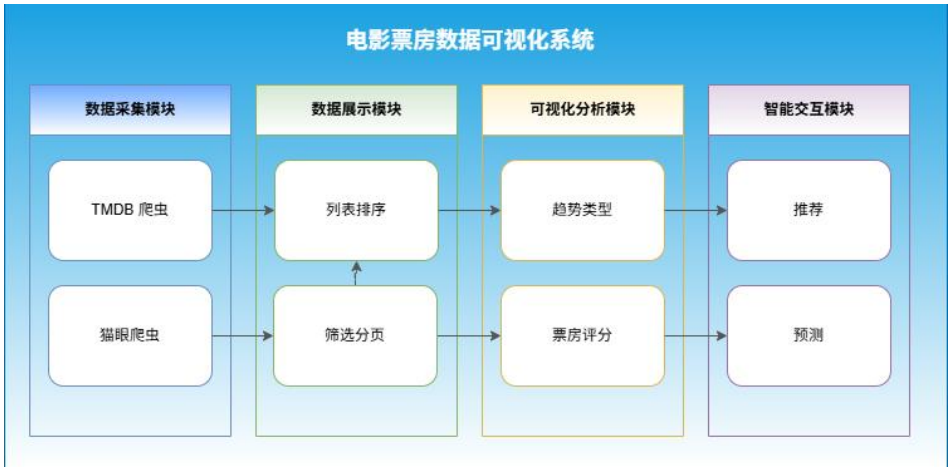


图 3 架构流程图

前后端分离的优势：前端页面通过 CDN 引入 ECharts 和 TailwindCSS，无需构建工具即可运行；后端专注于数据接口和业务逻辑，前端专注交互和展示，双方通过 API 文档约定数据格式，便于并行开发和后续维护。

4.2. 后端 API 设计

采用 RESTful 风格设计，共 11 个接口。接口按功能分为三类：数据查询类（列表、筛选、详情）、统计分析类（趋势、票房、类型、评分分布）、系统管理类（缓存管理）和智能交互类（智能助手）。统一返回格式 {"code": 200, "data": {...}, "msg": "..."}。

接口	方法	功能	缓存时间
/api/movies	GET	电影列表查询（筛选、排序、分页）	5 分钟
/api/filters	GET	获取类型/年份筛选项	1 小时
/api/year-trend	GET	每年电影数量趋势	1 小时
/api/revenue-trend	GET	每年平均票房（按年份排序）	1 小时
/api/stats	GET	统计数据	无
/api/rating-distribution-grouped	GET	评分分布（0.5 分档）	1 小时
/api/genre-distribution	GET	类型分布（全部类型）	1 小时
/api/movie/{id}	GET	电影详情	1 小时
/api/movie/{id}/keywords	GET	电影关键词	24 小时
/api/cache/clear	POST	清除缓存	—
/api/cache/status	GET	缓存状态	—
/api/agent/recommend	POST	智能助手（推荐/预测）	—

接口设计说明：高频接口（列表、筛选项、趋势）设置缓存以减少数据库压力；统计数据接口不设缓存保证实时性；关键词接口缓存 24 小时因数据变化极慢；智能助手接口不设

缓存因每次对话内容不同。

4.3. 智能助手时序图

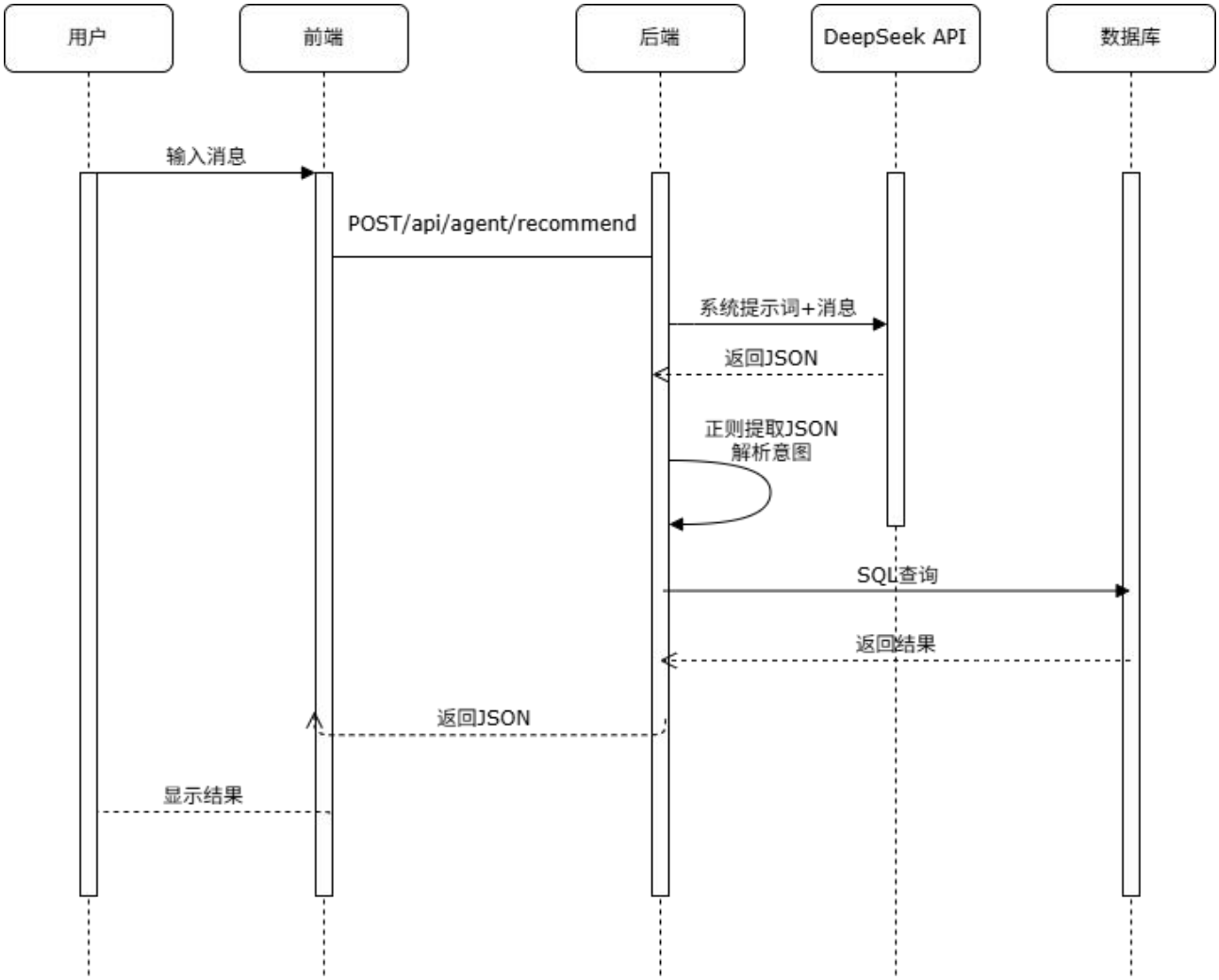


图 4 智能助手时序图

时序图流程说明：用户发送消息后，后端先调用 DeepSeek API 解析用户意图，提取类型、年份、预算等结构化参数，然后根据解析结果（推荐/预测）执行对应的 SQL 查询或预测计算，最后将结果返回前端渲染。

4.4. 类图

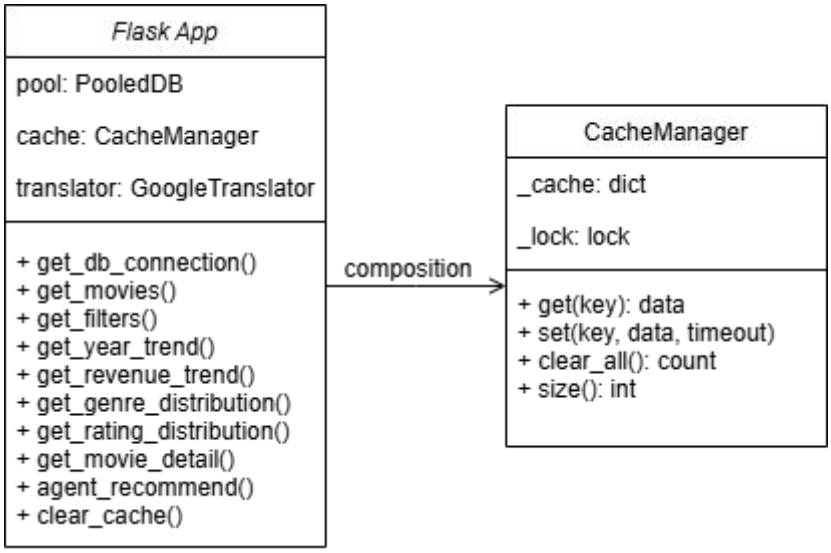


图 5 系统类图

类图说明：Flask App 组合了 CacheManager 作为缓存管理器，二者是强依赖关系。CacheManager 使用 _cache 字典存储数据和过期时间，_lock 保证多线程环境下缓存操作的线程安全。接口方法均返回 JSON 格式数据，由前端统一渲染。

4.5. 缓存设计说明

配置项	说明
缓存策略	内存缓存（Memory Cache），非 Redis
缓存 Key 生成	根据请求参数（page、per_page、sort 等）生成唯一键
过期策略	TTL（Time To Live），到期自动删除
清理时机	数据更新时手动调用 /api/cache/clear 清除全部缓存
线程安全	使用 threading.Lock 保证多线程并发安全

引入缓存的原因：列表查询、筛选项、趋势统计等接口数据变化频率低但访问频率高（用户切换排序、翻页等操作频繁），每次请求都执行 SQL 查询会造成数据库不必要的压力。引入内存缓存后，相同请求直接返回缓存数据，响应时间从平均 200ms 降至 5ms 以内。

4.6. 预测模块设计

用户输入"预测 2027 年科幻电影票房，预算 8000 万"后，系统首先调用硅基流动 DeepSeek API 解析自然语言，返回结构化参数：
{"intent": "predict", "predict_genre": "科幻", "predict_budget": 80000000, "predict_year": 2027}。
后端根据 intent 字段进入预测流程，调用预测函数执行计算。

预测采用加权系数相乘模型，核心公式为：

$$P = B \times 2.5 \times G \times Y \times R \times S \times F$$

公式中，P 为预测票房，B 为预算，G 为类型系数，Y 为年份系数，R 为评分系数，S 为预算系数，F 为随机波动。

类型系数 G 由系统内置的经验值表决定，不同类型取值不同：
科幻 2.0、奇幻 1.9、动作 1.8、动画 1.7、冒险 1.6、恐怖 1.5、喜剧 1.4、犯罪 1.3、家庭 1.3、剧情 1.2、爱情 1.2、纪录片 0.4。

年份系数 Y 反映市场年度增长，以 2010 年为基准每年增长 3%，计算公式为：

$$Y = 1 + (T - 2010) \times 0.03$$

评分系数 R 反映评分与票房的正相关关系，以 7 分为基准，计算公式为：

$$R = 0.5 + (S/10) \times 1.5$$

预算系数 S 反映大预算电影的规模效应，以 1000 万美元为基准，计算公式为：

$$S = 1 + log10(B/10000000) \times 0.1$$

随机波动 F 模拟市场不确定性，取值 0.85~1.15 随机生成。预测结果受预算 0.5 倍下限和 15 倍上限约束。

计算示例：2027 年科幻电影，预算 8000 万，预期评分 7 分。基础票房为 8000 万×2.5=2 亿。

$$\text{年份系数 Y} = 1 + (2027 - 2010) \times 0.03 = 1.51$$

$$\text{评分系数 R} = 0.5 + (7/10) \times 1.5 = 1.5$$

$$预算系数\ S = 1 + \log10(8) \times 0.1 = 1.208$$

$$P = 2\ 亿 \times 2.0 \times 1.51 \times 1.55 \times 1.208 \approx 11.3\ 亿$$

$$ROI = (11.3\ 亿 - 8000\ 万)/8000\ 万 \times 100\% \approx 1312\%。$$

5. 编码实现

5.1. 开发环境

- 操作系统：Windows 11
- 编程语言：Python 3.13
- Web 框架：Flask 2.3.x
- 数据库：MySQL 8.0
- 前端技术：原生 JavaScript + ECharts 5 + TailwindCSS
- 开发工具：VS Code + Postman + Git

5.2. 运行环境

- 后端：Python 3.13 + Flask + PyMySQL + DBUtils + deep-translator
- 数据库：MySQL 8.0（InnoDB 引擎，utf8mb4 字符集）
- 前端：现代浏览器（Chrome / Firefox / Edge），无需构建工具
- 第三方服务：硅基流动 DeepSeek API（自然语言解析）

5.3. 关键代码分析

1. 数据库连接池

每次请求都创建新连接开销很大，连接池复用连接提升性能。

```
from dbutils.pooled_db import PooledDB
pool = PooledDB(
    creator=pymysql,
    maxconnections=20,
    mincached=3,
```

```
    maxcached=10,
    blocking=True,
    host='127.0.0.1',
    user='root',
    password='20060321',
    database='movie_db',
    charset='utf8mb4',
    cursorclass=pymysql.cursors.DictCursor
)


def get_db_connection():
    """从连接池获取数据库连接"""
return pool.connection()
```

2. 内存缓存

内存缓存减少重复数据库查询。不同接口设置不同过期时间：列表 5 分钟、筛选项 1 小时、关键词 24 小时。

```
class CacheManager:
    def __init__(self):
        self._cache = {}
        self._lock = Lock()


    def get(self, key):
        with self._lock:
            if key in self._cache:
                data = self._cache[key]
                if data['expire'] > time.time():
                    return data['data']
                else:
                    del self._cache[key]
```

```
            return None

    def set(self, key, data, timeout=CACHE_TIMEOUT):
        with self._lock:
            self._cache[key] = {
                'data': data,
                'expire': time.time() + timeout
            }
```

3. 智能助手 JSON 解析

AI 返回的 JSON 常被 markdown 包裹，需先去除标记再用正则提取纯净 JSON，避免解析报错。

```
def call_deepseek_api(user_message):
    # ... 调用 API ...
    raw_content = result["choices"][0]["message"]["content"]

    # 清理 markdown 代码块
    cleaned = raw_content.strip()
    if cleaned.startswith("```json"):
        cleaned = cleaned[7:]
    elif cleaned.startswith("```"):
        cleaned = cleaned[3:]
    if cleaned.endswith("```"):
        cleaned = cleaned[:-3]

    # 正则提取 JSON 对象
    import re
    json_match = re.search(r'\{[\s\S]*\}', cleaned)
    if json_match:
        cleaned = json_match.group()
```

```
return json.loads(cleaned)
```

4. ECharts 图标初始化（柱状图为例）

ECharts 初始化后通过 setOption 配置图表，渐变着色使柱子颜色随数值变化，图表的 tooltip 和 dataZoom 交由 ECharts 内部处理。

```
function renderCharts(data) {
    const years = data.map(item => item.year);
    const counts = data.map(item => item.count);

    const barChart = echarts.init(document.getElementById('trend-chart'));
    barChart.setOption({
        tooltip: { trigger: 'axis' },
        grid: { left: 55, right: 25, top: 10, bottom: 30 },
        xAxis: { type: 'category', data: years },
        yAxis: { type: 'value', name: '数量' },
        series: [{
            type: 'bar',
            data: counts,
            barWidth: '30%',
            itemStyle: {
                color: new echarts.graphic.LinearGradient(0, 0, 0, 1, [
                    { offset: 0, color: '#4361ee' },
                    { offset: 1, color: '#b8d4fa' }
                ])
            }
        }]
    });
}
```

5.4. 主要界面截图

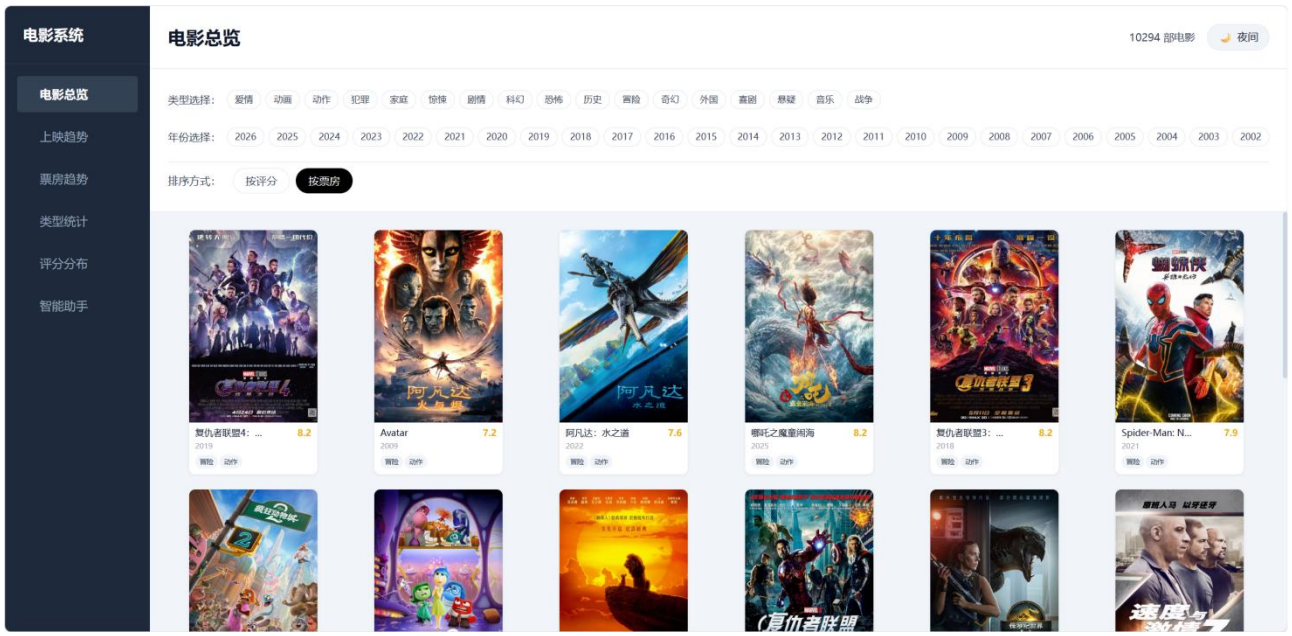


图 6 电影总览页

电影总览页以卡片网格展示电影，顶部提供类型多选、年份多选和排序切换（按评分/按票房）功能。卡片包含海报、片名、评分和年份。页面支持滚动触底无限加载（每次 24 条），图片采用懒加载策略，无海报时显示"无封面"占位文字。卡片悬停时有上浮和阴影效果。

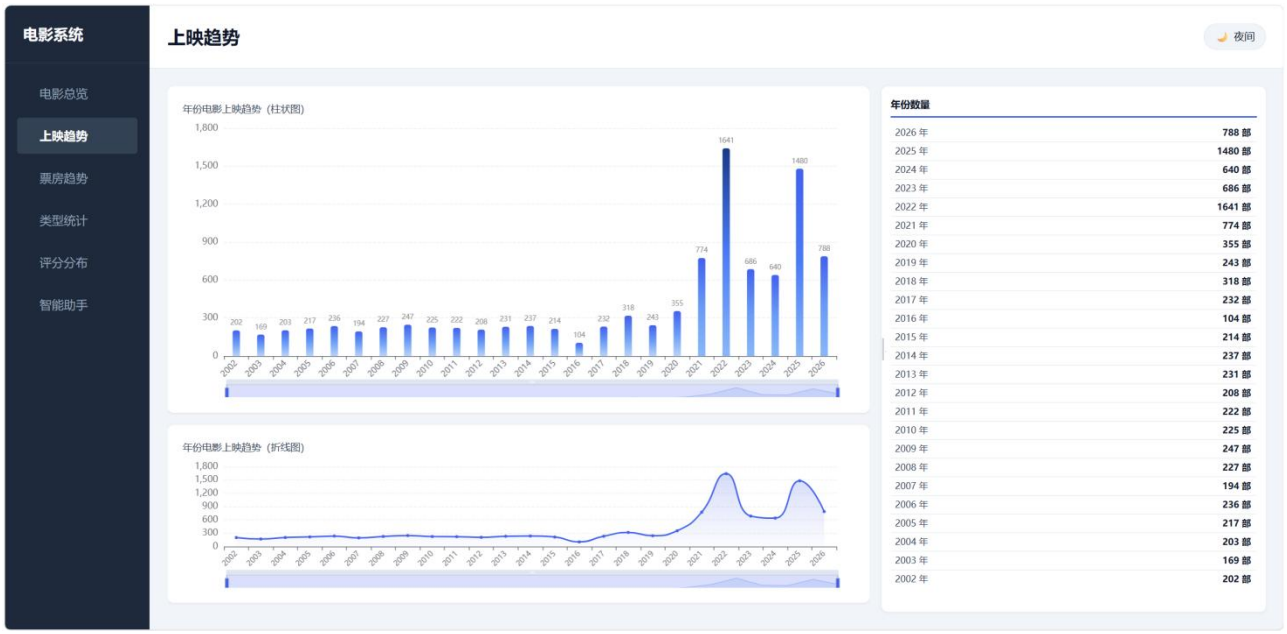


图 7 上映趋势页

上映趋势页左侧为主图表区域，上方为柱状图展示各年份电影数量分布，颜色采用蓝色渐变

且数量越多颜色越深；下方为折线图展示趋势变化，曲线平滑并带有面积填充。右侧侧边栏列出各年份的具体数量统计列表，支持拖拽调整侧边栏宽度。图表支持数据缩放滑块，方便查看特定年份区间。

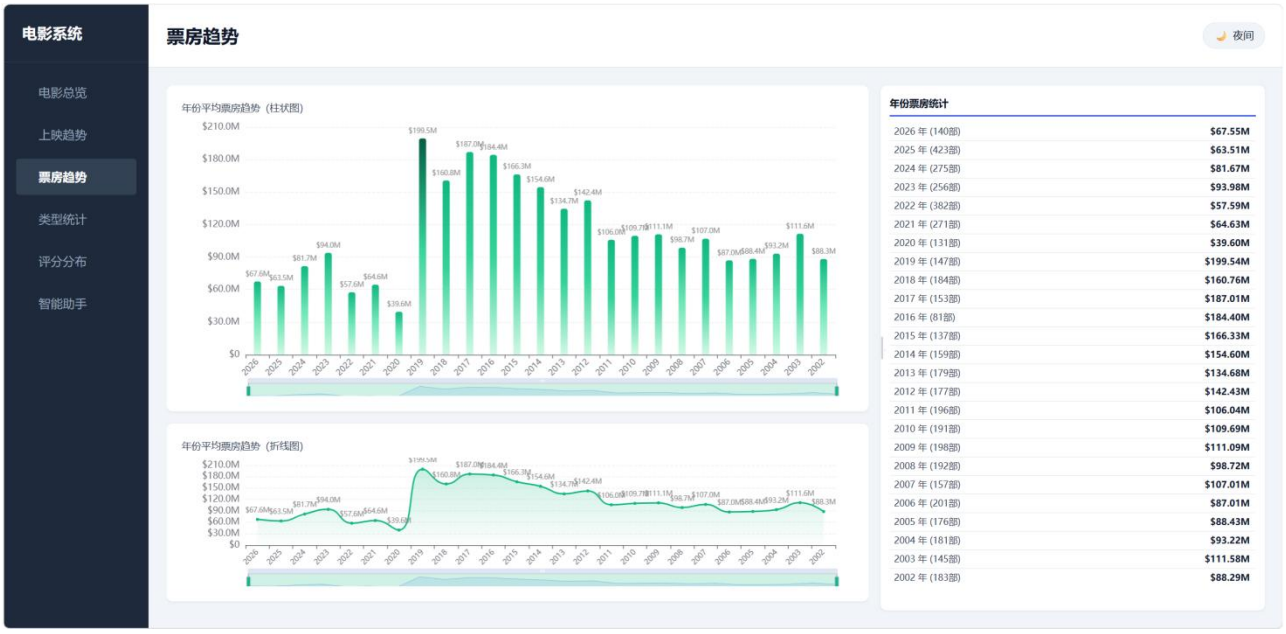


图 8 票房趋势页

票房趋势页展示各年份的平均每部电影票房（而非总票房），以消除年份间电影数量差异带来的偏差。柱状图采用绿色渐变，折线图带有面积填充。右侧侧边栏显示每年电影数量及平均票房金额（以百万美元为单位），数据已过滤票房为 0 的记录且限制至少 3 部电影才显示。



图 9 类型统计页

类型统计页采用黄色系主题，上方为环形图展示各类型占比，下方为柱状图展示各类型具体数量。配色从浅黄到深棕渐变，显示全部类型而非仅 TOP N，标签在空间充足时展示类型名称。

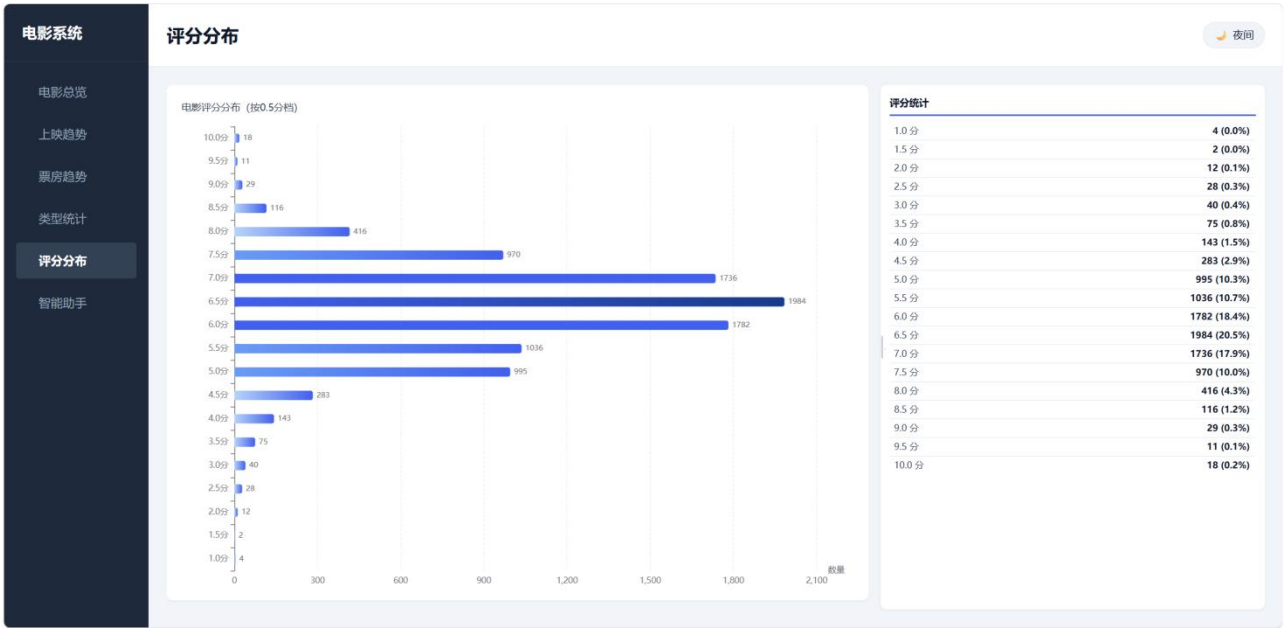


图 10 评分分布页

评分分布页采用横向条形图展示各评分档位（按 0.5 分档聚合）的电影数量，Y 轴为评分档位（如"8.5 分"），X 轴为电影数量，标签显示在条形右侧便于阅读。右侧侧边栏列出各档位的具体数量和占比，配色采用蓝色渐变且数量越多颜色越深。

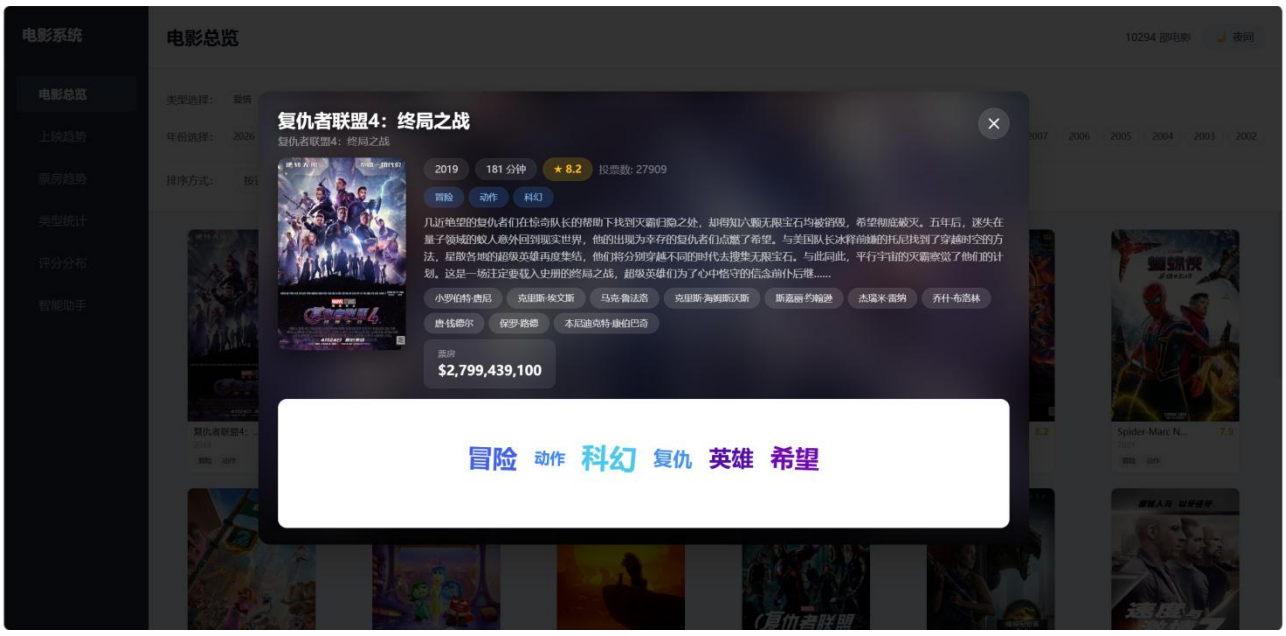


图 11 影详情弹窗

电影详情弹窗采用模糊背景加清晰封面双层布局，背景使用海报图模糊 30px 并降低亮度。弹窗包含电影标题、年份、片长、评分、类型标签、中文简介、演员列表和票房信息。底部为关键词词云区域，10 个词随机分配大小和颜色，悬停时放大。

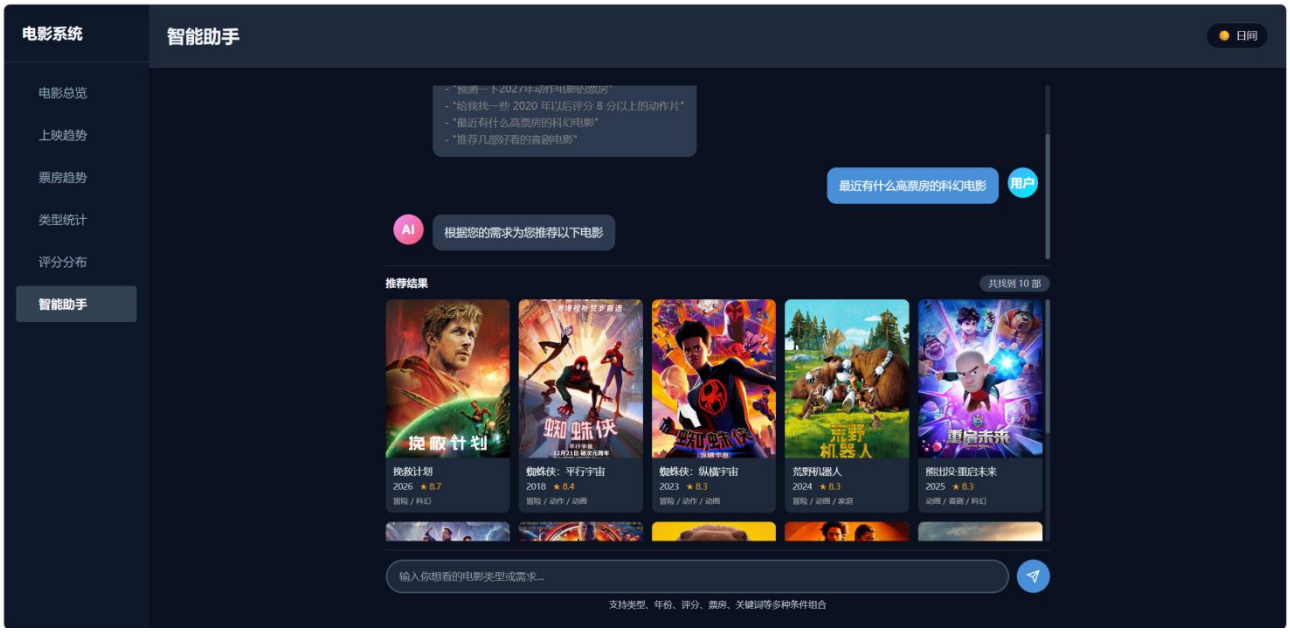


图 12 智能助手页面-推荐电影

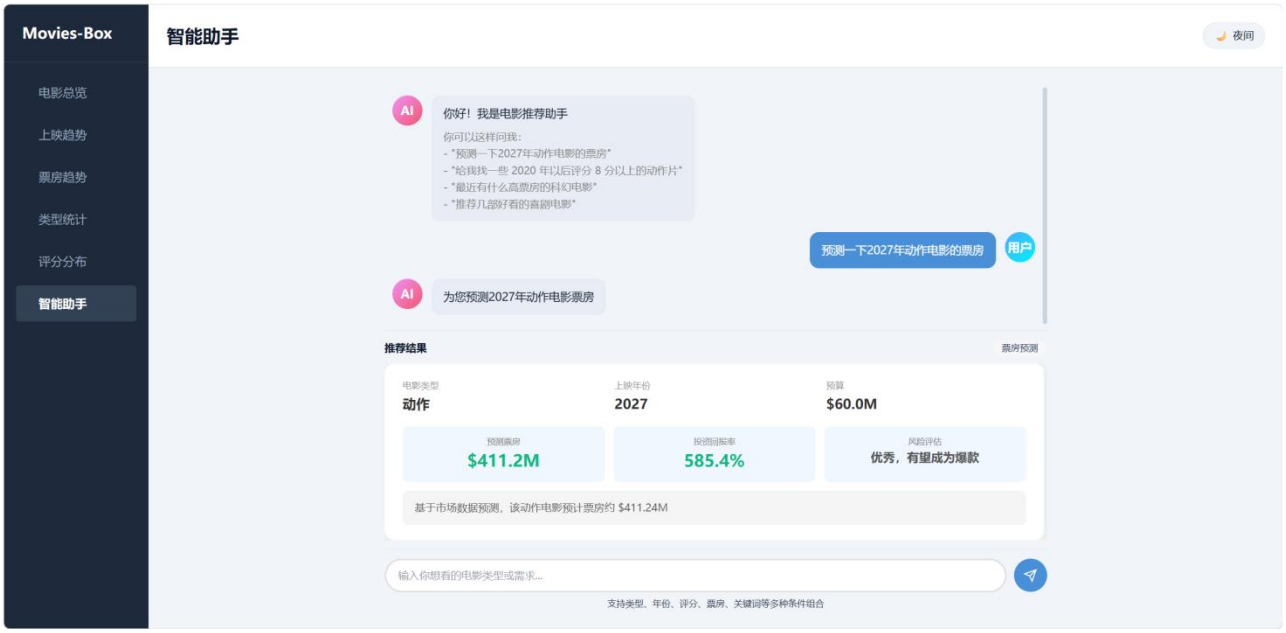


图 13 智能助手页面-预测票房

智能助手页面为聊天界面风格，用户输入自然语言需求，系统调用 DeepSeek API 解析意图后返回推荐电影列表或票房预测结果。推荐结果以卡片网格展示，包含海报、片名、年份、

评分和类型；预测结果以卡片形式展示预算、预测票房、投资回报率和风险评估。

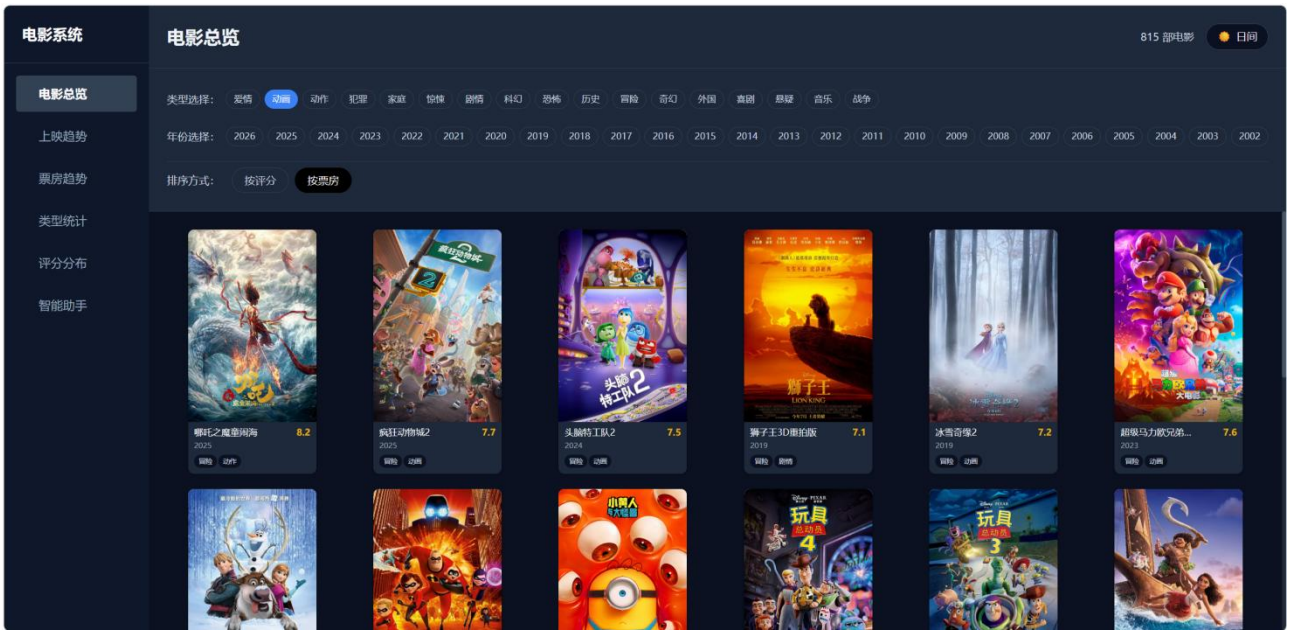


图 14 夜间模式

页面右上角提供夜间模式切换开关。开启后，背景、卡片、文字、边框等所有元素颜色同步切换为暗色系，图表背景也随之更新，用户偏好通过 localStorage 持久化存储。

6. 程序调试情况

6.1. 数据采集阶段

- 问题 1：猫眼字体反爬
  猫眼电影的票房、评分等数据在 HTML 中显示为乱码符号，浏览器中看到的"9.2"在爬虫获取的源码中是加密字符。
- 解决方案：网站使用@font-face 定义自定义字体文件，将真实数字映射为特定编码字符。通过 fontTools 解析.woff 字体文件，比对字形坐标建立"新编码→真实数字"的动态映射表。同时注意字体文件 URL 和字符编码会随页面刷新变化，采用"以形补数"的思路——字符形状（坐标轮廓）不变，通过比对坐标对象识别数字。

- 问题 2：猫眼反爬综合策略：User-Agent 校验：requests 不加 headers 直接请求返回 403 → 伪装真实浏览器 User-Agent，携带完整请求头。
  IP 频率限制：短时间大量请求触发风控 → 控制请求间隔 2-3 秒，使用代理池轮换 IP
  Cookie 验证：无有效 Cookie 无法获取数据 → 手动登录后提取 Cookie。

数据异步加载：票房数据通过 AJAX 加载 → 抓包分析 XHR 请求，找到真实数据接口。

6.2. 后端开发阶段

- 问题 3：类型字段格式不匹配导致 JSON_CONTAINS 查询失败
  智能助手查询时使用 JSON_CONTAINS 函数报错"Invalid JSON text in argument 1"。
- 原因分析：数据库中类型_标准化字段存储的是逗号分隔的字符串（如"喜剧,动作"），而非 JSON 数组格式（如["喜剧","动作"]），JSON_CONTAINS 要求字段为有效 JSON 格式。
- 解决方案：将查询方式从 JSON_CONTAINS 改为 LIKE 模糊匹配，使用类型_标准化 LIKE %s 进行类型筛选。

- 问题 4：Python 导入缺失导致运行时崩溃
  运行时连续报错 name 'datetime' is not defined 和 name 'random' is not defined。
- 原因分析：predict_revenue 函数中使用了 datetime.now().year 和 random.random()，但文件顶部未导入相应模块。
- 解决方案：在 app.py 顶部补全所有依赖导入：from datetime import datetime、import random、import math。

6.3. 前端开发阶段

- 问题 5：演员数据显示[object Object]
  电影详情弹窗中演员列表显示为[object Object]字符串。
- 原因分析：数据库中演员字段存储为 JSON 对象数组（如[{"name":"Leonardo","character":"Cobb"}]），前端直接对数组进行字符串拼接导致调用默认 toString()方法。
- 解决方案：后端在返回详情数据前，解析 JSON 数组并只提取 name 字段组成的列表传给前端。

- 问题 6：词云图一直显示"生成关键词中"
  ECharts WordCloud 插件无法加载，词云区域持续显示加载提示。
- 原因分析：ECharts WordCloud 插件通过 CDN 加载，网络不稳定时加载失败，且 CDN 地址更新导致 404。
- 解决方案：放弃 CDN 依赖，改用纯 CSS+JavaScript 实现。为每个关键词创建 span 标签，随机分配字体大小（18-38px）、颜色和旋转角度（-12 度到+12 度），利用 flex-wrap

实现自动换行布局，悬停时放大动画。加载速度从数秒降为即时，无需额外网络请求。

- 问题 7：图表渲染空白
  切换导航时图表区域显示空白，无任何内容。
- 原因分析：图表容器在初始化时未设置固定高度，或被父容器隐藏时渲染导致尺寸为 0。
- 解决方案：为图表容器设置 min-height: 240px，使用 ResizeObserver 监听容器尺寸变化并调用 chart.resize()重绘，确保图表始终有正确尺寸。

6.4. 智能助手阶段

- 问题 8：AI 返回 JSON 被 markdown 包裹导致解析失败
  DeepSeek API 返回的内容带有```json 标记，json.loads()直接解析报错。
- 解决方案：先去除开头的 json 或，再去除结尾的```，最后用正则 re.search(r'\{[\s\S]*\}', cleaned)提取纯净 JSON 对象，确保无论 AI 返回什么格式都能正确解析。

- 问题 9：前后端数据契约不一致
  后端返回 is_prediction: True 标识预测模式，但前端未检查该字段，仍按推荐模式渲染，显示"共找到 0 部"。
- 原因分析：前后端开发时未同步约定数据格式，前端 sendAgentMessage 函数仅处理了 movies 数组，未处理预测结果。
- 解决方案：前端添加 if (data.data.is_prediction === true)判断分支，预测模式调用 displayPrediction 函数渲染卡片，推荐模式调用 displayRecommendations 渲染列表。

- 问题 10：API 调用失败时系统无响应
  DeepSeek API 因网络波动超时或密钥问题失败时，用户请求无返回，界面卡住。
- 解决方案：实现备用解析机制（fallback_parse），当 API 调用失败时通过关键词匹配降级处理——检测"预测"、"类似"、"推荐"等关键词和类型关键词，提取年份、预算等信息，保证系统在 API 不可用时仍能提供基础服务。

7. 系统总结

7.1. 系统特点

- 双源数据整合：系统整合了 TMDB 全球电影数据和猫眼中国票房数据，兼顾全球视野和本土特色，数据覆盖 2002-2026 年共 10,294 条记录，包含海报、演员等完整字段。

- 多维度可视化：基于 ECharts 实现了上映趋势、票房趋势、类型统计、评分分布四个维度的可视化分析，图表配色统一、交互流畅（缩放、悬停提示），侧边栏可拖拽调整宽度。
- 自然语言交互：集成硅基流动 DeepSeek API，用户可通过自然语言获得电影推荐和票房预测，大幅降低使用门槛。系统具备备用解析机制，API 不可用时自动降级到关键词匹配。
- 完整的用户体验：支持夜间模式持久化、无限滚动加载、图片懒加载、电影详情弹窗、词云展示，交互细节打磨完善。
- 性能优化：数据库连接池（PooledDB，最大 20 连接）+ 内存缓存（TTL 过期策略）+ 前端懒加载，接口响应迅速，万余条数据下浏览流畅。

7.2. 存在问题

- 预测模型过简：票房预测仅基于类型系数、预算规模和年份增长趋势，未考虑档期（春节档、暑期档等）、导演历史票房、演员号召力、宣发投入、同期竞争等关键因素。模型更适合"趣味估算"而非商业决策工具。
- 数据未自动更新：当前数据采集为一次性脚本，无法自动获取新上映电影的信息，系统数据会随时间滞后。需要建立定时任务每周自动爬取更新。
- 爬虫稳定性不足：猫眼反爬策略频繁更新（字体文件不定期更换、接口加密逻辑调整），当前爬虫需要持续维护才能保持可用。
- 定性不足：猫眼反爬策略频繁更新（字体文件不定期更换、接口加密逻辑调整），当前爬虫需要持续维护才能保持可用。

7.3. 改进意见

- 预测模型升级：引入档期系数（春节档 2.0、暑期档 1.6、普通档 0.8）、导演历史平均票房、演员热度指数、营销预算投入等特征，建立更完善的回归模型或机器学习模型。
- 数据自动更新：使用 APScheduler 或 Celery 建立定时任务（如每周日凌晨 2 点），自动调用 TMDB API 获取增量数据，确保系统数据与 TMDB 保持同步。
- 爬虫稳定性增强：建立字体文件变化监控机制，自动检测新字体并更新映射表；使用代理池和动态 UA 轮换；考虑使用 Selenium 无头模式降低被检测概率。

8. 课程设计总结

通过本次课程设计，我深入体验了一个完整软件项目的开发全流程——从需求分析、数据采集、数据库设计、前后端开发到系统部署。以下是几点深刻体会：

1. 数据采集远比想象中复杂。初以为爬虫就是简单的 requests.get()，实际面对的是猫眼动

态字体反爬、Cookie 验证、IP 封禁等多层防护。破解字体反爬的过程让我理解了"以形补数"的思路——不依赖变化的编码，而是通过字符形状（字体坐标）来建立映射，这种抽象思维对解决问题很有帮助。

2. 前后端分离架构需要严谨的数据契约。开发智能助手时，后端返回的 is_prediction 字段前端没有检查，导致预测结果被当作推荐列表渲染，显示了"共找到 0 部"。这个问题让我意识到前后端必须事先约定好接口数据格式，并在开发过程中保持同步。

3. 大语言模型集成需要健壮的容错机制。AI 返回的 JSON 可能被 markdown 包裹、可能包含额外文字、可能因网络波动超时。仅靠 json.loads()直接解析是不够的，需要完善的清理逻辑（正则提取）+ 备用方案（关键词匹配降级），才能保证系统可用性。

4. 做工程不是写 Demo。许多细节决定了系统的可用性——连接池避免反复创建连接、缓存减少数据库压力、懒加载提升首屏速度、夜间模式增强用户体验。这些"非功能需求"往往是决定一个系统能否真正投入使用的关键。

5. 善用 AI 工具但要保持批判性思维。本次开发使用了 DeepSeek API 辅助编码，但在过程中多次指出了 AI 回答中的错误——包括未确认数据库字段格式就使用 JSON_CONTAINS、导入缺失导致运行时崩溃、清理逻辑不完整导致 JSON 解析失败等。AI 能提高效率，但真正的系统设计、问题诊断和决策仍需开发者具备扎实的技术判断力。

通过本次实践，我对前后端分离架构、数据库优化、可视化技术、大语言模型集成和软件工程方法论有了更深入的理解和掌握。

9. 参考文献

期刊

[1] 谢彦南,杨呈敏. 电影票房数据采集与可视化研究[J].信息与电脑(理论版),2021,33 (23):176-178.

[2] 杨应浩.基于 Python 的电影信息爬取与数据可视化分析[J].新型工业化,2021,11(7):71+73. DOI:10.19335/j.cnki.2095-6649.2021.7.032.

[3] 高巍,孙盼盼,李大舟.基于 Python 爬虫的电影数据可视化分析[J].沈阳化工大学学报,2020,34(1):73-78.

图书

[1] 《Python 数据分析项目全程实录》

[2] 《Python 项目开发实战》

[3] 《数据可视化》